Predicting 30-Day Hospital Readmission

Kaggle Challenge Report

Task: predict whether a patient is readmitted within 30 days of discharge

Metric: ROC-AUC on predicted probabilities

Final public leaderboard score: 0.74685

1. Overview

The task is to predict, for each of about 33,000 test hospital stays, the probability that the patient is readmitted within 30 days of discharge (readmit_30d). Only about 10% of stays are positive (7,755 of 76,975 training stays, 10.07%), and submissions are scored with ROC-AUC. The data is a synthetic hospital network of 110,000 index stays spread over 12 linked tables.

We turned the tables into one row per stay, cleaned the raw data, and compared 13 model configurations from 10 algorithm families with the same 5-fold cross-validation. Regularised gradient boosting was best (LightGBM, cross-validated AUC 0.7521). Our best leaderboard result, 0.74685, came from a rank blend of two LightGBM models and a logistic regression (cross-validated AUC 0.7527). Every model family we tried ended between 0.696 and 0.753, and neither extra features, feature selection, further tuning nor blending moved the cross-validated score by more than 0.0007, which suggests the data itself limits what can be predicted.

2. Data pipeline

2.1 Tables used and dropped

We used all 12 modelling tables and dropped none. The spine is index_encounters, which holds every train and test stay (109,787 rows); train.csv and test.csv only supply the labels and the split. Each other table covers most stays, costs little to aggregate, and boosted trees ignore uninformative columns, so we kept every table instead of guessing which ones to discard. Our later experiments (Section 7) showed that the extra columns built from labs, vitals, medications, procedures and transfers in the third feature set added almost nothing, which suggests a smaller subset would probably do about as well, although we did not test dropping a whole table. icd10_lookup was used only to add the chapter and chronic flag to diagnoses, and data_dictionary.csv served as a reference.

2.2 Joins

index_encounters was left-joined to patients on patient_id and to hospitals on hospital_id. Every many-row table was first aggregated to one row per encounter_id and then left-joined on encounter_id. prior_utilization is keyed by patient, so for each stay we kept only visits dated before that stay's admission and aggregated them per patient before joining. Labels were attached from train.csv only at modelling time, and train and test went through exactly the same code.

2.3 Aggregation of the many-row tables

Table	Rows	Stays covered	How it was aggregated per stay
diagnoses	451,976	100%	Counts (all, chronic, not present on admission), primary diagnosis and its ICD chapter, counts per ICD chapter, 0/1 flags for the 63 most common 3-character codes, Charlson score
procedures	19,547	17.1%	Count, first code, days from admission and to discharge, distinct codes
lab_results	1,668,783	100%	Per test (13 tests): last, mean, min, max, count, last minus first, hours before discharge of the last value, flag of the last value, slope; totals of abnormal flags
vitals	2,267,367	100%	Per vital (6 types): last, mean, min, max, std, count, last minus first; last-24-hour mean/min/max; counts of readings outside clinical thresholds
medications	447,278	98.0%	Counts, distinct names, continued at discharge, IV count, 0/1 flag per drug class and per common drug name, start and stop timing
unit_transfers	211,097	100%	Number of moves, first and last unit, hours per unit, ICU flag, timing of the last move and the last ICU stay
discharge_planning	97,552	88.9%	Used as is (at most one row per stay) plus a row-present flag
prior_utilization	714,062	99.5% of patients	ED, inpatient and outpatient counts in the last 30, 90, 180 and 365 days and in total, days since the last visit of each type, same-hospital versus other-hospital counts

Table 1. Many-row tables and how each was summarised. Rows are counted before duplicates were removed.

3. Final dataset

The final modelling table has 109,787 rows (76,975 training and 32,812 test stays) and 463 columns (444 numeric, 19 categorical). Four more columns, target-encoded versions of hospital, attending, primary diagnosis and first procedure code, are added at model time inside each training fold, so they are not stored in the table. The complete list is in the attached final_columns.csv with one row per column, giving its source table, how it was created, the feature-set version it first appeared in, its type and its percentage of missing values.

Source table	Columns	Version 1	Version 2	Version 3
lab_results	128	81	3	44
diagnoses	88	82	6	0
medications (and drug classes)	87	28	2	57
vitals	68	42	1	25
prior_utilization	33	22	2	9
index_encounters	17	17	0	0
unit_transfers	14	8	1	5
patients	8	8	0	0
procedures	7	2	0	5
discharge_planning	6	6	0	0
hospitals	4	4	0	0
derived from several tables	3	0	3	0
Total	**463**	**300**	**18**	**145**

Table 2. Final columns by primary source table and by the feature-set version in which they were added.

Version 1 (300 columns) was the first full pipeline. Version 2 added 18 columns (a Charlson comorbidity score, a LACE-style readmission score, ratios and counts of out-of-range labs). Version 3 added 145 columns describing the patient's state at discharge (timing and trends of labs and vitals, drug names, procedure and transfer timing, same-hospital prior visits). The two high-cardinality identifiers, attending_id (about 1,000 levels) and the first procedure code, are not fed to the models raw; their target-encoded versions are used instead.

4. Data quality

The tables come from different source systems and were not cleaned. The table below lists what we found and what we did.

Problem found	Evidence	How we handled it
Exact duplicate rows	diagnoses 902, labs 3,331, vitals 2,265, medications 893, prior visits 1,284	Dropped
Lab units mixed within one test	Albumin g/L 18,308 vs g/dL 37,277; creatinine umol/L 42,578 vs mg/dL 168,327; glucose mmol/L 29,666 vs mg/dL 232,432; hemoglobin g/L 62,388 vs g/dL 128,567	Converted to one unit per test (g/L x 0.1, umol/L / 88.4, mmol/L x 18.016)
Implausible lab values (unit slips, typos)	5,194 results (0.31%) outside a physiological range after conversion	Set to missing; the count per stay is kept as a feature
Temperature in Fahrenheit and Celsius	66,538 of 378,097 temperature readings (17.6%) above 45	Converted from Fahrenheit to Celsius
Impossible vital signs	369 readings (heart rate 189, SpO2 180), for example SpO2 of 0 or heart rate of 300	Set to missing
ICD-10 codes missing the decimal point	182,326 codes (40.3%); 124,211 rows did not match the lookup table	Decimal point inserted after the third character, so they match the lookup
Drug names spelled many ways	122 spellings for 52 drugs (case and spacing)	Lower-cased and trimmed before building drug features
Same procedure coding system under two names	CPT 17,238 rows vs CPT-4 2,309 rows	Treated as one system
Placeholder text instead of missing	race '?' 1,103; sex 'U' 56; 'Not Available' or 'Not Mapped' in admission type 3,209, admission source 4,351, discharge disposition 1,174	Converted to missing
Inconsistent categories	insurance type had 10 spellings of 5 values (MEDICARE, Medicare)	Standardised to title case
Implausible patient and stay values	179 birth years before 1910; stays of 0 days or over 120 days; follow-up scheduled more than 60 days away	Set to missing
Dates in several formats	ISO in encounters, labs and transfers; MM/DD/YYYY HH:MM in vitals; DD-Mon-YYYY in procedures; YYYYMMDD integers in medications	Parsed with one format per table

Table 3. Data quality problems and fixes.

5. Missing values

Many gaps are informative, because they reflect what was measured or done for a patient. The readmission rates below are from the training stays. The causes are our interpretation, since we cannot see how the data was generated.

Where	Missing	Likely cause	Readmission rate
Admission weight	87.0%	Only recorded for a subset of stays	12.6% if recorded, 9.7% if not
Attending specialty	46.4%	Not captured by the source system; looks random	10.2% if missing, 10.0% if not
Procedures	No row for 82.9% of stays	No procedure performed (a real zero, not an error)	10.6% without, 7.3% with
Discharge planning	No row for 11.1%; follow-up days missing in 41.8% of existing rows	Planning not done or not recorded	10.6% without a row, 10.0% with
Selective labs	Albumin ordered in 37.2% of stays, HbA1c 16.0%, troponin 14.4%, BNP 12.3%, lactate 9.7%, INR 6.2%	Tests are ordered when clinically indicated	12.7% to 15.5% if ordered vs 8.5% to 9.5% if not (INR 10.8% vs 10.0%)
Lab abnormal flag	21.2% of results	Flag not filled in by some systems	Not checked
Medication stop date	61.2% of rows	Drug still active, open-ended	Not checked
Patient registration fields	Insurance 17.2%, marital status 9.0%, language 7.1%, deprivation index 4.7%, zip 4.0%, race 1.5%	Gaps at registration	Insurance 9.4% if missing, 10.2% if not; others about equal
Prior visits	No prior rows for 0.5% of patients	First-time patients	4.8% without, 10.1% with

Table 4. Main sources of missing values.

Because missingness often carries information (stays with many specialised labs ordered had clearly higher readmission), we did not use heavy imputation. What we did:

- Boosted-tree models (LightGBM, XGBoost, HistGradientBoosting) handle missing values natively, so we left them missing and added flags such as weight_missing, spec_missing and dp_present.
- When a table has no row for a stay (no procedures, no prior visits), count columns are set to 0 because that is the true value, while timing columns stay missing.
- For models that cannot take missing values (logistic regression, SVM, LDA, MLP, Naive Bayes, KNN, forests) we used median imputation with a missing indicator per column and a separate "missing" category for text columns, all fitted on the training part of each fold only. On a 20,000-row sample, median imputation scored slightly better than a constant fill for a decision tree (AUC 0.693 vs 0.689).
- The count of tests ordered (the lab count columns) lets the models see which tests were ordered at all.

6. Models

6.1 How we tuned and evaluated

- All models use the same 5-fold stratified cross-validation on the 76,975 training stays (shuffled, fixed seed), so scores are comparable and predictions from different models can be blended.
- Every imputer, scaler and encoder is fitted on the training part of each fold only. The target encoding of hospital, attending, primary diagnosis and first procedure code uses an inner 5-fold scheme for the training rows, so no row sees its own label.
- Boosting models were tuned by hand over regularisation strength, because early runs showed the signal is weak and shallow, heavily regularised trees work best. A final grid of nine LightGBM settings (leaves, minimum rows per leaf, L1 and L2 penalties, column sampling) was run to check that we had reached the optimum. The number of rounds came from early stopping on the validation fold (a small optimism in the cross-validated scores); the final models use 1.1 times the mean best round count.
- The other families were tuned by random search on a 30,000-row sample with 3-fold cross-validation (3 to 30 configurations per model), and the best configuration was re-scored with the full 5-fold protocol. Naive Bayes and KNN were tuned over scalers (standard, robust, min-max, quantile, power transform) and feature selection by the F-test.
- We did not resample or re-weight the classes. ROC-AUC depends only on how patients are ranked, so class weights mainly change calibration.
- Final models were refitted on all training stays with 2 or 3 random seeds averaged. Blend weights were picked from out-of-fold predictions, not from the leaderboard.

6.2 The three best models

Model	Key hyperparameters	Cross-validated AUC	Public leaderboard
LightGBM (v2)	learning rate 0.04, 4 leaves, min 400 rows per leaf, 20% of columns per tree, 80% row sampling, L2 = 60, 700 rounds, 3 seeds; target encoding; 318 columns	0.7521	0.74589
XGBoost (v5)	eta 0.04, depth 3, min child weight 20, 80% rows, 30% columns, lambda 30, alpha 1, 363 rounds, 3 seeds; target encoding	0.7502	0.74397
CatBoost (v3)	depth 5, learning rate 0.06, L2 = 10, 30% of columns per split, 64 borders, 445 rounds, 2 seeds; native categorical handling	0.7476	0.74337

Table 5. The three best single models. LightGBM first used a looser setting (15 leaves, 290 rounds; CV 0.7494, leaderboard 0.74326).

6.3 All models compared

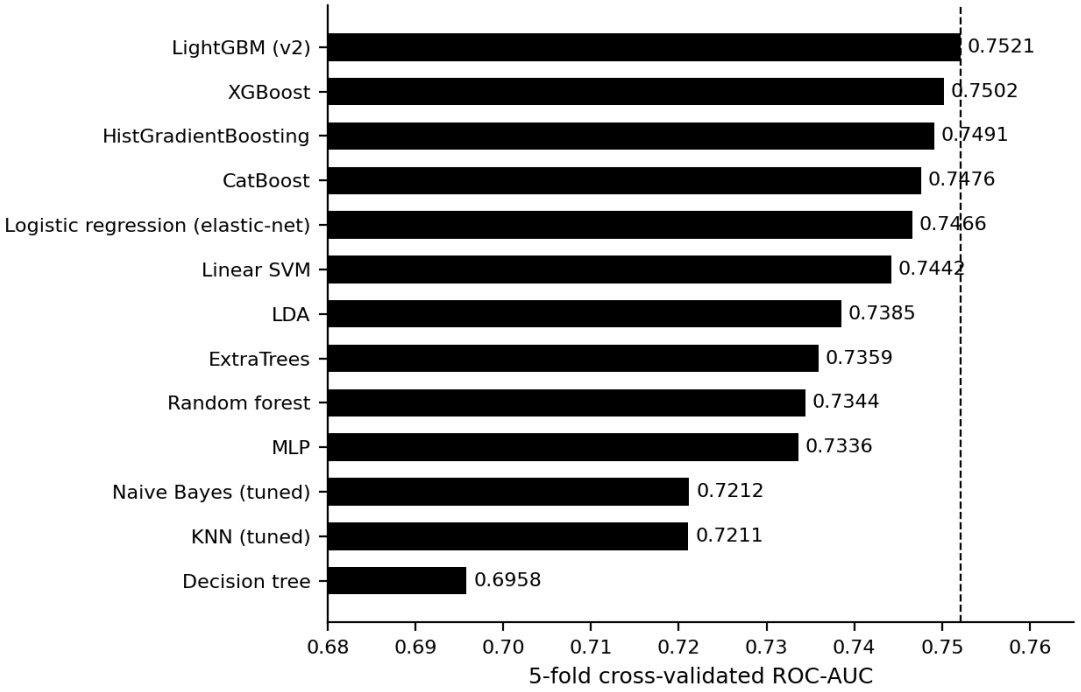


Figure 1. Cross-validated ROC-AUC of every model on the same folds. The dashed line is the best single model.

Model	Tuned settings	CV AUC
HistGradientBoosting	learning rate 0.03, depth 3, 8 leaves, min 200 rows per leaf, L2 = 10, 30% of columns (30 configs)	0.7491
Logistic regression (elastic-net)	C = 0.012, l1 ratio 0.2, standardised inputs (12 configs)	0.7466
Linear SVM (calibrated)	C = 0.0066 (8 configs)	0.7442
Linear discriminant analysis	shrinkage 0.5 (6 configs)	0.7385
ExtraTrees	400 trees, depth 14, min 40 rows per leaf, 20% of columns (3 configs)	0.7359
Random forest	500 trees, depth 16, min 40 rows per leaf, 20% of columns, 80% row sampling (20 configs)	0.7344
MLP (neural network)	layers 128-64-32, L2 = 0.1, learning rate 0.003 (12 configs)	0.7336
Naive Bayes	quantile transform to normal, 80 best columns, smoothing 1e-6 (untuned: 0.6968)	0.7212
KNN	standard scaler, 80 to 200 best columns, k = 800, distance weighted (untuned: 0.6844)	0.7211
Decision tree	depth 5, min 200 rows per leaf (untuned: 0.6988)	0.6958

Table 6. Other models (cross-validation only, not submitted).

6.4 Ensembles

We combined models by rank averaging, which fits a rank-based metric, and rescaled the blend to the probability scale of the LightGBM predictions so the file contains probabilities.

Ensemble	Cross-validated AUC	Public leaderboard
v4: 0.8 LightGBM + 0.2 logistic regression	0.7525	0.74682
v6: 0.4 LightGBM (version 2 columns) + 0.4 LightGBM (version 3 columns) + 0.2 logistic regression	0.7527	0.74685
v7, Kaggle notebook: LightGBM (version 2 and version 3 columns, 3 leaves) + logistic regression, weights 0.4 / 0.4 / 0.2	0.7528	0.74679
Best greedy blend over all 11 models (not submitted)	0.7527	not submitted
Stacking all 11 models with logistic regression (not submitted)	0.7518	not submitted

Table 7. Ensembles. Picking weights on half of the stays and scoring on the other half gave no real gain over the fixed 0.8 / 0.2 blend (0.7512 vs 0.7513).

7. Results and discussion

7.1 Leaderboard result

Our final public leaderboard score is 0.74685 (version 6). Our submissions scored between 0.74326 and 0.74685, and the submission written by the final Kaggle notebook scored 0.74679. With our best score at that time (0.74682) we were ranked 56th, and the top score on the board was 0.74964. The public score uses about 30% of the test stays and the final ranking uses the other 70%, so we chose our submissions by cross-validation and not by the public score. Across our submissions the leaderboard score ran about 0.006 below the cross-validated AUC, and the order of our submissions mostly matched the order of their cross-validated scores.

7.2 Which model was best and why

Regularised gradient boosting was best, with LightGBM at 0.7521. Boosting handles the mix of 19 categorical and 444 numeric columns, the many missing values and interactions (for example between diagnosis and hospital) without manual work. The signal in this data is weak, so what mattered most was regularisation. Moving from the first setting (15 leaves, light regularisation) to 4 leaves with 400 rows per leaf and strong L2 raised the leaderboard score from 0.74326 to 0.74589, while the new columns added in the same step contributed only about 0.0003 of cross-validated AUC. Logistic regression (0.7466) came close, which says that much of the signal is additive. Bagged trees and the neural network were about 0.015 lower, and KNN, Naive Bayes and the single tree were lowest, probably because they suffer from the many weak columns.

In the first LightGBM run the most important columns were the maximum creatinine, the primary diagnosis, the hospital, the number of prior visits, the deprivation index, the number of chronic diagnoses and age.

7.3 Why we think the score is limited by the data

- Thirteen configurations from ten algorithm families landed between 0.696 and 0.753, and the top five are within 0.006 of each other.
- A final grid of nine LightGBM settings scored between 0.7513 and 0.7522 (the best, 3 leaves, was 0.7522 against 0.7521 for our setting), and target-encoding ten categorical columns instead of four did not help.
- Blending LightGBM with XGBoost, CatBoost, logistic regression or the weaker models gained at most 0.0007, because the strong models are 95% to 99% correlated in their rankings.
- Adding 145 columns on the patient's state at discharge gave 0.7518 against 0.7521 without them, and keeping only the 60, 100 or 150 most important columns gave 0.7504, 0.7510 and 0.7514.

We cannot prove that no better model exists, but because very different methods and very different feature sets arrive at the same score, we believe most of the remaining error is noise in the data (it is synthetic) and not something a more complex model could remove.

7.4 Limitations

- Differences below about 0.001 AUC are within the noise of cross-validation and of the 30% public leaderboard, so the order of our top submissions should not be over-read.

- Early stopping used the validation fold, which makes the boosting scores very slightly optimistic. The same applies to the weights of the blends.
- Results move in the fourth decimal between runs and machines: for example CatBoost scored 0.7476 here and 0.7479 when we re-ran it in a notebook on Kaggle.
- Some families were tuned with few configurations on a sample (for example ExtraTrees with 3), and AdaBoost was started but not completed, so it is not reported.

7.5 What we would try next

- Repeat the cross-validation with several fold splits and many more seeds, to separate real differences of 0.001 from noise and to reduce the variance of the final model.
- Tune the target-encoding smoothing and add interactions between hospital and diagnosis, since hospital and primary diagnosis were among the strongest columns.
- Try a neural network with embeddings for the high-cardinality identifiers (attending, hospital, diagnosis codes), a different family from the trees we used.
- Try an additive model with splines (a GAM-style logistic regression) and a pairwise ranking objective, which we did not test. Our grid over LightGBM settings was flat, so more tuning of the same model is unlikely to help.

Appendix A. Submission history

ID	Model	CV AUC	Public AUC
v1	LightGBM, 3 seeds, 15 leaves, 290 rounds	0.7494	0.74326
v2	LightGBM, regularised, version 2 columns, target encoding	0.7521	0.74589
v3	CatBoost, 2 seeds	0.7476	0.74337
v4	Blend 0.8 LightGBM (v2) + 0.2 logistic regression	0.7525	0.74682
v5	XGBoost, 3 seeds, target encoding	0.7502	0.74397
v6	Blend 0.4 LightGBM (v2) + 0.4 LightGBM (v3 columns) + 0.2 logistic regression	0.7527	0.74685
v7	Kaggle notebook: blend 0.4 LightGBM (v2 columns, 3 leaves) + 0.4 LightGBM (v3 columns, 3 leaves) + 0.2 logistic regression	0.7528	0.74679

Table 8. All submissions to the leaderboard.

Appendix B. Attached files

- **final_columns.csv**: every final column with its source table, how it was created, the feature-set version, its type and percentage of missing values (467 rows, including the four target-encoded columns).
- **Notebook**: kaggle_readmission_notebook.ipynb runs from the raw CSV files through cleaning, the three feature sets, the model comparison, tuning and the final blend to a submission file. Run on Kaggle, it reproduced our cross-validated scores (for example LightGBM 0.7521 and 0.7518, XGBoost 0.7502, logistic regression 0.7469). Its submission file (v7) scored 0.74679 on the public leaderboard.
- **Code**: the GitHub repository contains the scripts that run from the raw CSV files to each submission (features.py, features_v2.py, features_v3.py, the model scripts and the experiment log).